

# Estructuras de Datos

## Clase 11 – Análisis de algoritmos (Tiempo de ejecución)



Dr. Sergio A. Gómez  
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación  
Universidad Nacional del Sur  
Bahía Blanca, Argentina

# Motivaciones

- Eficiencia: Capacidad del software de correr usando la mínima cantidad de recursos de hardware.
- La herramienta de análisis que usaremos consiste de caracterizar el “tiempo de ejecución” de algoritmos y operaciones de estructuras de datos (con uso de espacio de memoria también de interés).
- Objetivo: Una aplicación debe correr lo más rápidamente posible.

# Factores que afectan el tiempo de ejecución

- Aumenta con el tamaño de la entrada de un algoritmo
- Puede variar para distintas entradas del mismo tamaño
- Depende del hardware (velocidad del reloj, procesador, cantidad de memoria, tamaño del disco, ancho de banda de la conexión a la red)
- Depende del sistema operativo
- Depende de la calidad del código generado por el compilador
- Depende de si el código es compilado o interpretado

# Cómo medir el tiempo de ejecución:

## (1) Estudio experimental

Con un algoritmo implementado, hacer varias corridas sobre distintas midiendo el tiempo de ejecución:

```
long horaInicio, horaTerminacion;

// Tiempo en nano segundos desde 01 enero 1970 00:00UTC
// 1 nanosegundo = 10(-9) segundos
horaInicio = System.nanoTime();

metodo(); // Ejecuto el método que quiero medir

horaTerminacion = System.nanoTime();

// 1 milisegundo = 10(-3) segundos
long tiempoCorridaMilisegundos = (horaTerminacion - horaInicio) / 1000000;

float tiempoCorridaSegundos = tiempoCorridaMilisegundos / 1000.0f;
```

# Cómo medir el tiempo de ejecución:

## (1) Estudio experimental

Hacer varias corridas sobre distintas entradas y realizar un gráfico de dispersión  $(n, t)$  con  $n$ =tamaño de la entrada natural y  $t$ =tiempo de corrida en segundos.

| $n$ | $t$  |
|-----|------|
| 10  | 132  |
| 20  | 462  |
| 30  | 992  |
| 40  | 1722 |
| 50  | 2652 |
| 60  | 3782 |

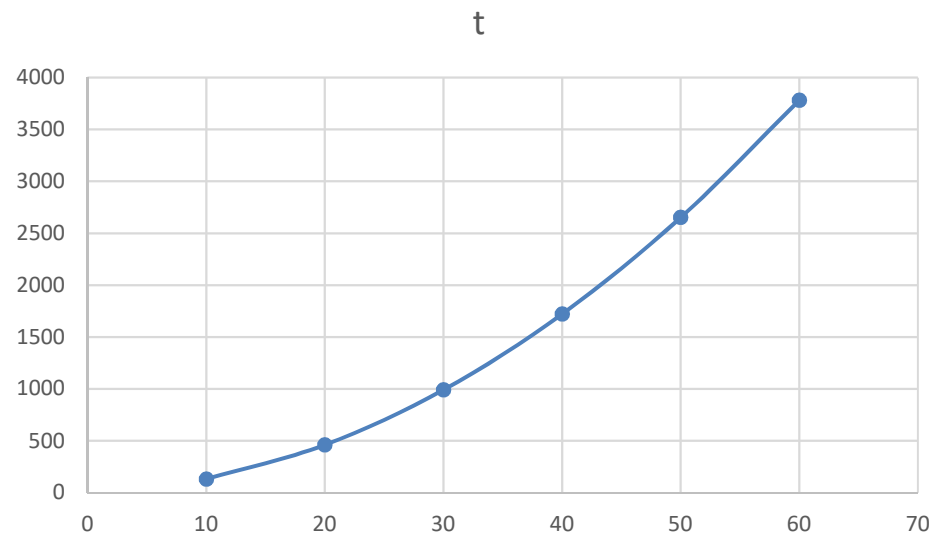


Gráfico de dispersión (scatter plot) de  $(n, t)$

# Cómo medir el tiempo de ejecución:

## (1) Estudio experimental

### Problemas:

- Se puede hacer con un número limitado de datos
- Dos algoritmos son incomparables a menos que hayan sido testeados en los mismos ambientes de hardware y software
- Hay que implementar el algoritmo para hacer el test

# El código compilado es más rápido que el código interpretado

```
prueba-millon.c - Code::Blocks 20.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
prueba-millon.c X
1 #include <stdio.h>
2 #include <time.h>
3
4 int main() {
5     clock_t inicio, fin;
6     double tiempo_transcurrido;
7
8     inicio = clock(); // Captura el tiempo de inicio
9
10    for (long long i = 0; i < 100000000; i++) {
11        // Operación vacía, solo iteramos
12    }
13
14    fin = clock(); // Captura el tiempo de finalización
15
16    // Conversión a segundos
17    tiempo_transcurrido = ((double)(fin - inicio)) / CLOCKS_PER_SEC;
18
19    printf("Tiempo de ejecución: %f segundos\n", tiempo_transcurrido);
20
21    return 0;
22 }
```

```
prueba-millon.py - C:/Users/sgomez/Desktop/prueba-millon.py (3.11.3)
File Edit Format Run Options Window Help
1 from time import time
2
3 inicio = time()
4 for i in range(100000000):
5     x = i
6 fin = time()
7 delta = fin - inicio
8 print("Tiempo (segundos): ", delta)
9 # Tiempo (segundos): 6.001970052719116
10
```

```
C:\Users\sgomez\Desktop\prueba-millon.exe
Tiempo de ejecución: 0.019000 segundos
Process returned 0 (0x0)   execution time : 3.968 s
Press any key to continue.
```

```
IDLE Shell 3.11.3
File Edit Shell Debug Options Window Help
Python 3.11.3 (tags/v3.11.3:f3909b8, Apr 4 2023, 23:49:59) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/sgomez/Desktop/prueba-millon.py =====
>>>
Tiempo (segundos): 5.946603536605835
>>>
```

```
C:\Users\sgomez\Desktop>java -jar bucle-10-millones.jar
Tiempo (seg): 0.004
```

Un programa que ejecuta, en un Core i7 de 7ma generación, un bucle for de 10 millones de iteraciones me tomó aprox. 6 segundos en Python (que es interpretado) versus aprox. 0,02 seg. en C (que es compilado) versus 0,006 segundos en Java en Eclipse versus 0,004 corriéndolo como .jar en la consola.

# Cómo medir el tiempo de ejecución:

## (2) Orden del tiempo de ejecución

- Toma en cuenta todas las posibles entradas
- Permite evaluar la eficiencia relativa de dos algoritmos independientemente del ambiente de hardware y software
- Puede realizarse estudiando una versión de alto nivel del algoritmo sin necesidad de implementarlo o hacer experimentos.



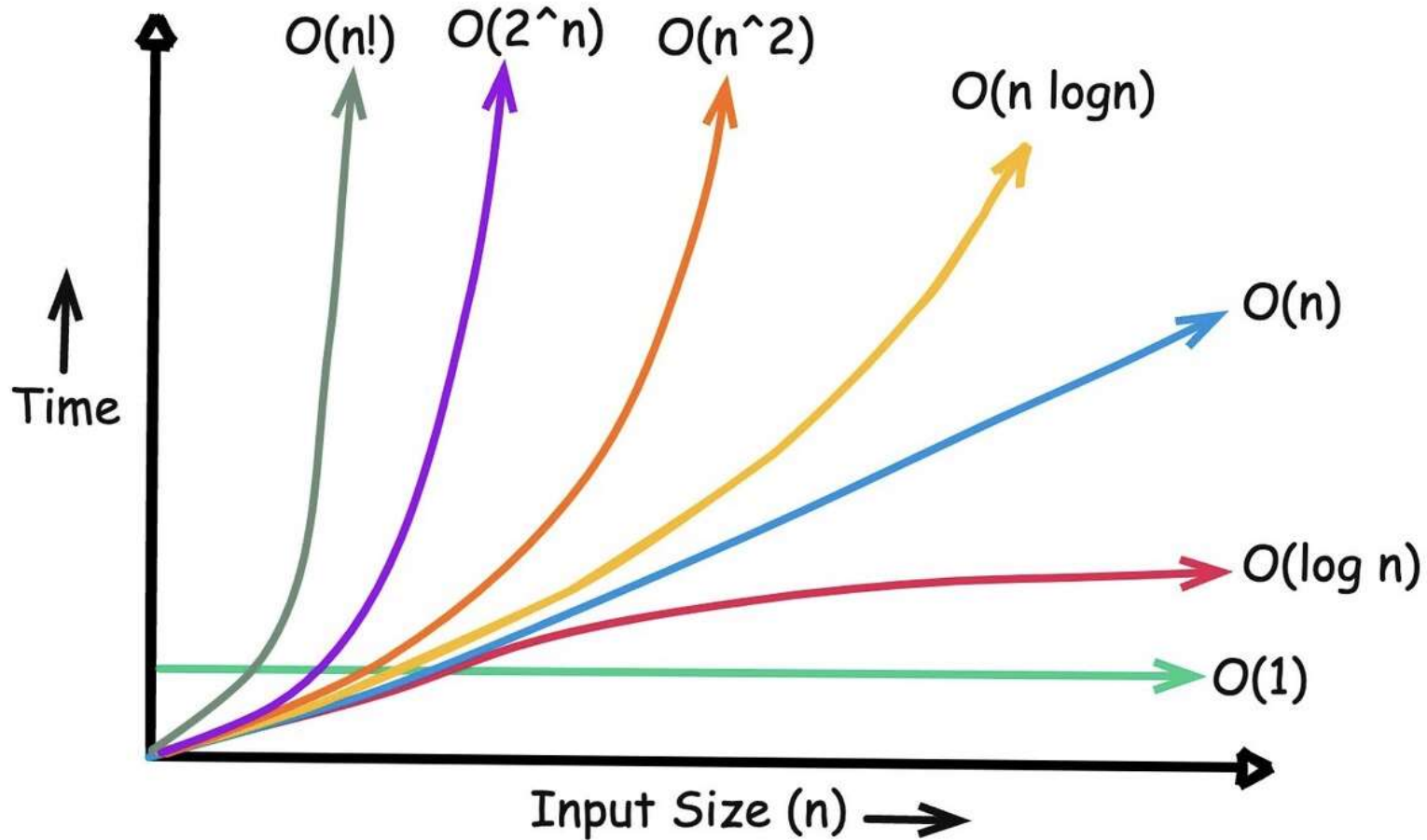
# Preliminares: Funciones

- Constante:  $f(n) = 1$
- Logaritmo:  $f(n) = \log_b(n)$  para  $b > 1$
- Lineal:  $f(n) = n$
- N-LogN:  $f(n) = n \log(n)$
- Cuadrática:  $f(n) = n^2$
- Cúbica:  $f(n) = n^3$
- Polinomial:  $f(n) = n^k$ , con  $k$  natural
- Exponencial:  $f(n) = a^n$  con  $a$  real positivo y  $n$
- Factorial:  $f(n) = n!$

# Funciones comparadas

| n     | 100 | log2(n)  | 3n    | nlog2(n) | n^2     | n^3     | 2^n      |
|-------|-----|----------|-------|----------|---------|---------|----------|
| 10    | 100 | 3.321928 | 30    | 33.21928 | 100     | 1000    | 1024     |
| 20    | 100 | 4.321928 | 60    | 86.43856 | 400     | 8000    | 1048576  |
| 30    | 100 | 4.906891 | 90    | 147.2067 | 900     | 27000   | 1.07E+09 |
| 40    | 100 | 5.321928 | 120   | 212.8771 | 1600    | 64000   | 1.1E+12  |
| 50    | 100 | 5.643856 | 150   | 282.1928 | 2500    | 125000  | 1.13E+15 |
| 60    | 100 | 5.906891 | 180   | 354.4134 | 3600    | 216000  | 1.15E+18 |
| 70    | 100 | 6.129283 | 210   | 429.0498 | 4900    | 343000  | 1.18E+21 |
| 80    | 100 | 6.321928 | 240   | 505.7542 | 6400    | 512000  | 1.21E+24 |
| 90    | 100 | 6.491853 | 270   | 584.2668 | 8100    | 729000  | 1.24E+27 |
| 100   | 100 | 6.643856 | 300   | 664.3856 | 10000   | 1000000 | 1.27E+30 |
| 200   | 100 | 7.643856 | 600   | 1528.771 | 40000   | 8000000 | 1.61E+60 |
| 1000  | 100 | 9.965784 | 3000  | 9965.784 | 1000000 | 1E+09   | 1.1E+301 |
| 2000  | 100 | 10.96578 | 6000  | 21931.57 | 4000000 | 8E+09   | #NUM!    |
| 10000 | 100 | 13.28771 | 30000 | 132877.1 | 1E+08   | 1E+12   | #NUM!    |

# Funciones comparadas



Tomada de internet

# Comparación de crecimiento asintótico en Análisis I

## ¿Por qué $\log(n)$ crece más lento que $n$ ?

Podemos considerar el límite:

$$\lim_{n \rightarrow \infty} \frac{\log(n)}{n}$$

Usando la regla de L'Hôpital, derivamos numerador y denominador:

$$\lim_{n \rightarrow \infty} \frac{\log(n)}{n} = \lim_{n \rightarrow \infty} \frac{1/n}{1} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$$

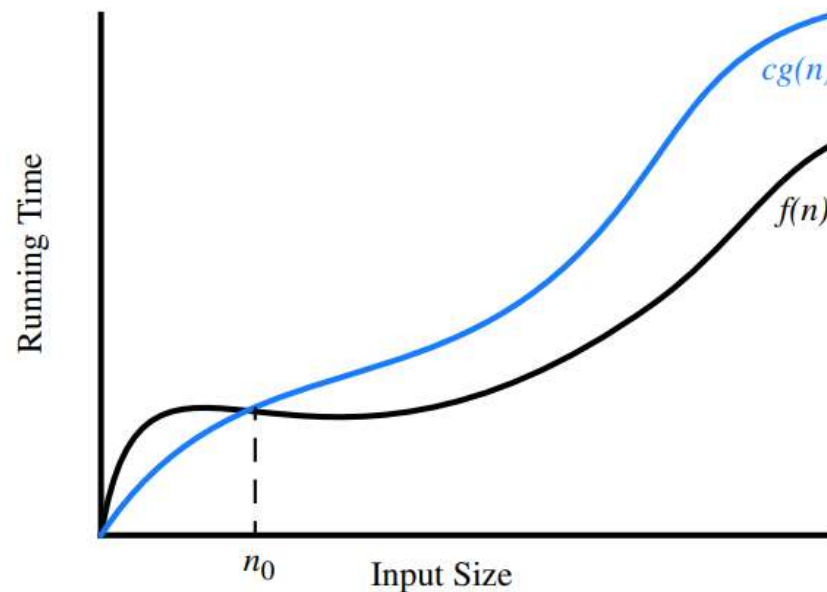
Como el límite es finito, esto sugiere que  $\log(n)$  crece a una tasa menor que  $n$ ,

# Comparación de crecimiento asintótico en complejidad computacional: Notación Big-Oh

Sean  $f(n)$  y  $g(n) : \mathbb{N} \rightarrow \mathbb{R}$

$f(n) = O(g(n))$  ssi existen  $c$  real con  $c > 0$  y  $n_0$  natural tal que  
 $f(n) \leq cg(n)$  para  $n \geq n_0$

Nota: “ $f(n) = O(g(n))$ ”  
se lee “ $f$  es del orden  
de  $g$ ” y/o “ $f$  es Big-Oh  
de  $g$ ”



# Propiedad útil

Si  $a \leq b$  y  $c \leq d$  entonces  $a + c \leq b + d$

Demostración:

Si  $a \leq b$  entonces  $b - a \geq 0$

Si  $c \leq d$  entonces  $d - c \geq 0$

La suma de dos números mayores o iguales a 0 es mayor o igual a 0, por lo tanto:

$$(b - a) + (d - c) \geq 0$$

Entonces  $b + d \geq a + c$ .

Luego,  $a + c \leq b + d$ .

Ejercicio: Mostrar que  $3n^2+2n+5$  es  $O(n^2)$

Proc: Hay que hallar  $c$  real y  $n_0$  natural tal que

$$3n^2+2n+5 \leq cn^2 \text{ para } n \geq n_0.$$

$$3n^2 \leq 3n^2 \text{ para } n \geq 0$$

$$2n \leq 2n^2 \text{ para } n \geq 0$$

$$5 \leq 5n^2 \text{ para } n \geq 1$$

$$\text{Luego } 3n^2+2n+5 \leq 3n^2 + 2n^2 + 5n^2 = (3+2+5)n^2 = 10n^2$$

Por lo tanto:  $c=10$

Para hallar  $n_0$  hay que usar el máximo de los  $n$ 's que vimos arriba que es 1

Luego  $n_0=1$ .

### Generalización para cualquier polinomio:

Prop: Mostrar que un polinomio en  $n$  de grado  $k$  es de orden  $n^k$ .

Dem: Sea  $P(n) = c_k n^k + c_{k-1} n^{k-1} + \dots + c_1 n + c_0$

$$c_k n^k \leq |c_k| n^k \text{ para } n \geq 0$$

$$c_{k-1} n^{k-1} \leq |c_{k-1}| n^k \text{ para } n \geq 0$$

...

$$c_1 n \leq |c_1| n^k \text{ para } n \geq 0$$

$$c_0 \leq |c_0| n^k \text{ para } n \geq 1$$

Entonces  $P(n) \leq \sum_{i=0..k} |c_i| n^k = (\sum_{i=0..k} |c_i|) n^k$  a partir de  $n_0=1$

Por lo tanto,  $P(n)$  es  $O(n^k)$



## Generalización: Reglas de la suma y el producto

- Regla de la suma:

Si  $f_1(n)$  es  $O(g_1(n))$  y

$f_2(n)$  es  $O(g_2(n))$  entonces

$f_1(n) + f_2(n)$  es  $O(\max(g_1(n), g_2(n)))$

- Regla del producto:

Si  $f_1(n)$  es  $O(g_1(n))$  y

$f_2(n)$  es  $O(g_2(n))$  entonces

$f_1(n) \times f_2(n)$  es  $O(g_1(n) \times g_2(n))$

Regla de la suma: Si  $f_1(n)$  es  $O(g_1(n))$  y  $f_2(n)$  es  $O(g_2(n))$  entonces  
 $f_1(n) + f_2(n)$  es  $O(\max(g_1(n), g_2(n)))$

Demostración:

Si  $f_1(n)$  es  $O(g_1(n))$  entonces existen  $c_1$  y  $n_1$  tales que  
 $f_1(n) \leq c_1 g_1(n)$  para todo  $n \geq n_1$

Si  $f_2(n)$  es  $O(g_2(n))$  entonces existen  $c_2$  y  $n_2$  tales que  
 $f_2(n) \leq c_2 g_2(n)$  para todo  $n \geq n_2$

Sea  $n_0 = \max(n_1, n_2)$ , entonces

$$f_1(n) \leq c_1 \max(g_1(n), g_2(n)) \text{ para } n \geq n_0$$

$$f_2(n) \leq c_2 \max(g_1(n), g_2(n)) \text{ para } n \geq n_0$$

Entonces  $f_1(n) + f_2(n) \leq (c_1 + c_2) \max(g_1(n), g_2(n))$  para  $n \geq n_0$ .

Luego  $f_1(n) + f_2(n)$  es  $O(\max(g_1(n), g_2(n)))$ , q.e.d.

### Regla del producto:

Si  $f_1(n)$  es  $O(g_1(n))$  y  
 $f_2(n)$  es  $O(g_2(n))$  entonces  
 $f_1(n) * f_2(n)$  es  $O(g_1(n)*g_2(n))$

### Demostración:

Si  $f_1(n)$  es  $O(g_1(n))$  entonces  $c_1$  y  $n_1$  tales que

$$f_1(n) \leq c_1 g_1(n) \text{ para todo } n \geq n_1$$

Si  $f_2(n)$  es  $O(g_2(n))$  entonces existen  $c_2$  y  $n_2$  tales que

$$f_2(n) \leq c_2 g_2(n) \text{ para todo } n \geq n_2$$

Sea  $n_0 = \max(n_1, n_2)$ , entonces

$$f_1(n) \leq c_1 g_1(n) \text{ para } n \geq n_0$$

$$f_2(n) \leq c_2 g_2(n) \text{ para } n \geq n_0$$

Entonces  $f_1(n) * f_2(n) \leq (c_1 g_1(n))(c_2 g_2(n)) =$   
 $(c_1 c_2)(g_1(n) g_2(n))$  para  $n \geq n_0$ .

Luego  $f_1(n) * f_2(n)$  es  $O(g_1(n)*g_2(n))$ , q.e.d.

## Big-Omega: Límite inferior asintótico

La function  $f(n)$  crece más o con la misma tasa que  $g(n)$ :

Big-Omega: Sean  $f(n)$  y  $g(n)$  funciones de los naturales en los reales.  $f(n)$  es  $\Omega(g(n))$  ssi existen  $c$  real positivo y  $n_0$  natural tal que  $f(n) \geq cg(n)$  para todo  $n \geq n_0$ .

Es decir, la curva de  $f(n)$  siempre se mantiene por encima o igual que  $cg(n)$ .

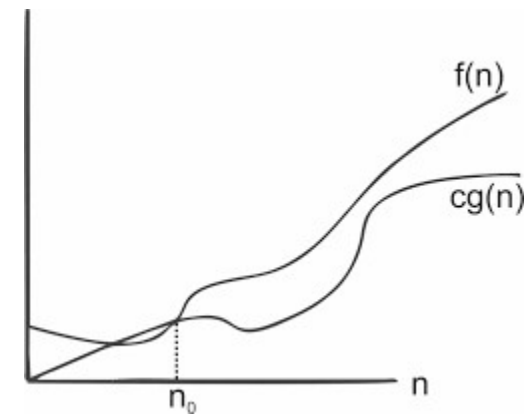
Ejemplo:  $3n\log(n) - 2n$  es  $\Omega(n\log(n))$

$$3n\log(n) - 2n = n\log(n) + 2n\log(n) - 2n$$

$$= n\log(n) + 2n\log(n) - 2n \cdot 1$$

$$= n\log(n) + 2n(\log(n) - 1) \geq n\log(n) \text{ para } n \geq 2.$$

Entonces tomamos  $c=1$  y  $n_0=2$ .

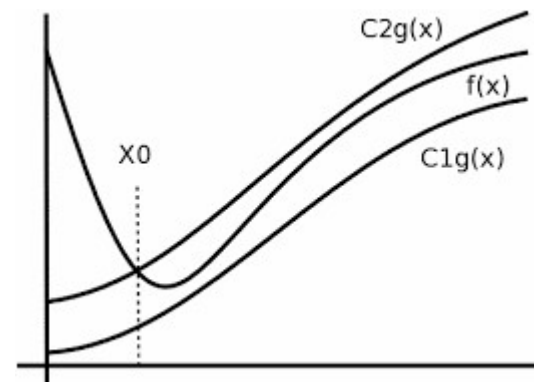


Tomada de  
internet

# Big-Theta

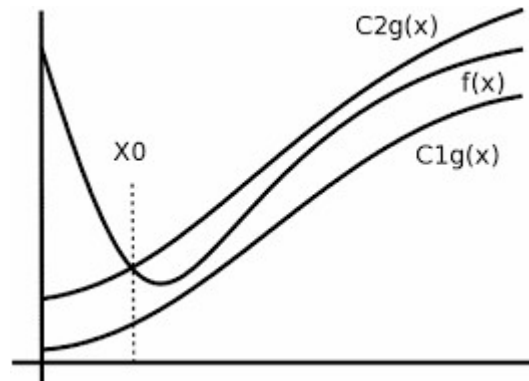
- Representa una cota ajustada de una función, la que está limitada inferior y superiormente por la misma función dentro de un factor constante.
- Big-Theta: Sean  $f(n)$  y  $g(n)$  funciones de los naturales en los reales.  $f(n)$  es  $\Theta(g(n))$  ssi  $f(n)$  es  $O(g(n))$  y  $f(n)$  es  $\Omega(g(n))$ .
- Nota: Big-Theta quiere decir  $c_1g(n) \leq f(n) \leq c_2g(n)$ . Por lo tanto, tienen un crecimiento asintótico equivalente.

Tomada de internet



# Big-Theta

- Recordamos: Big-Theta quiere decir  $c_1g(n) \leq f(n) \leq c_2g(n)$ .



- Ejemplo:  $3n \log(n) + 4n + 5\log(n)$  es  $\Theta(n \log(n))$   
 $3n\log(n) \leq 3n\log(n) + 4n + 5\log(n) \leq (3+4+5)n\log(n)$  para  $n \geq 2$ .

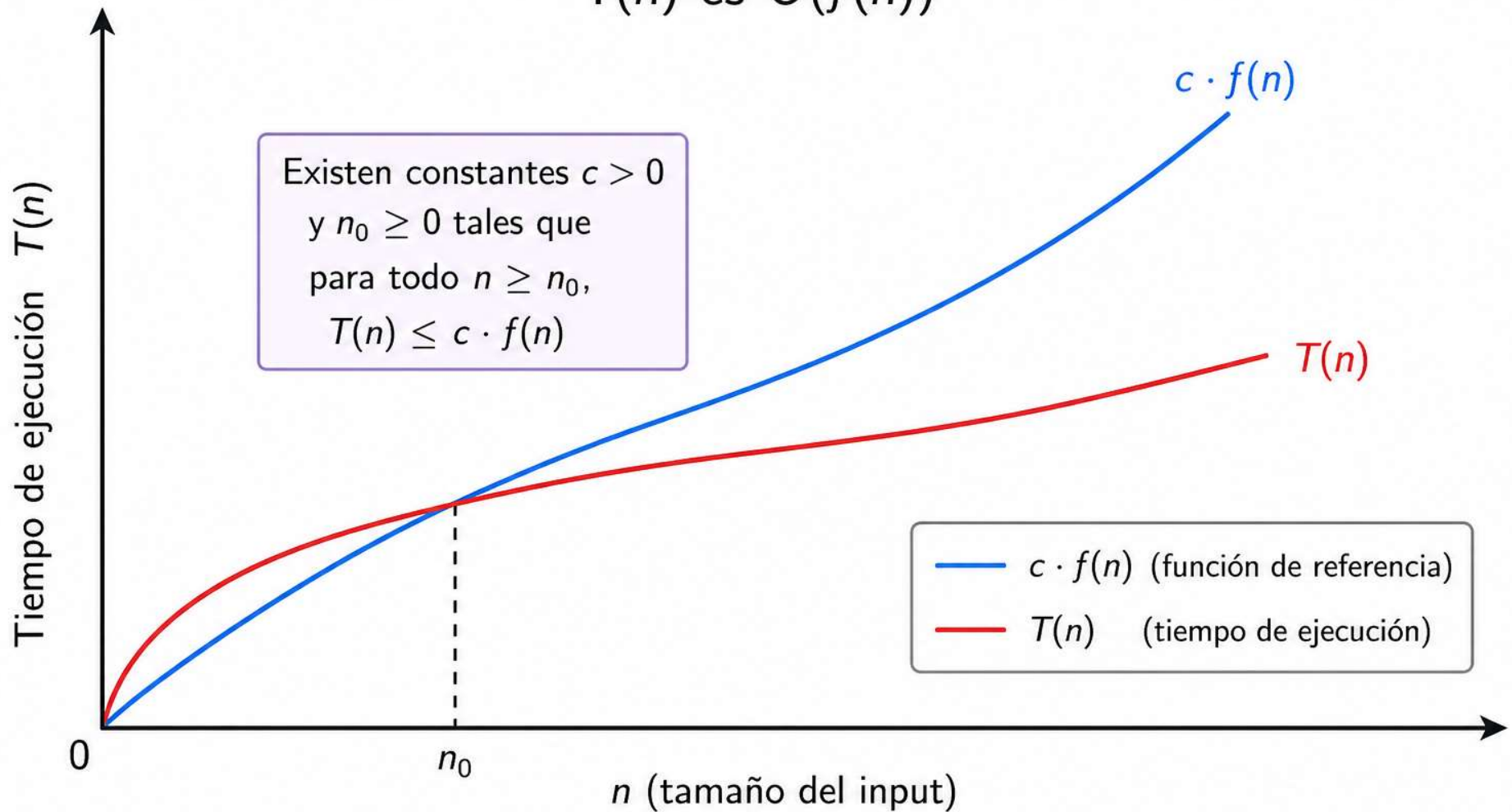
Tiempo de ejecución:

Cota superior del tiempo de un algoritmo A

Sea  $T_A(n)$  el tiempo de ejecución del peor caso de un algoritmo A definido sobre el tamaño  $n$  de la entrada de A.

$T_A(n) = O(f(n))$  si y solo si existen  $c$  y  $n_0$  tales que  $T_A(n) \leq cf(n)$  para todo  $n \geq n_0$ .

$T(n)$  es  $O(f(n))$



A partir de  $n_0$ , el tiempo de ejecución  $T(n)$  está acotado superiormente por una constante multiplicada por  $f(n)$ . Por lo tanto,  $T(n)$  es  $O(f(n))$ .



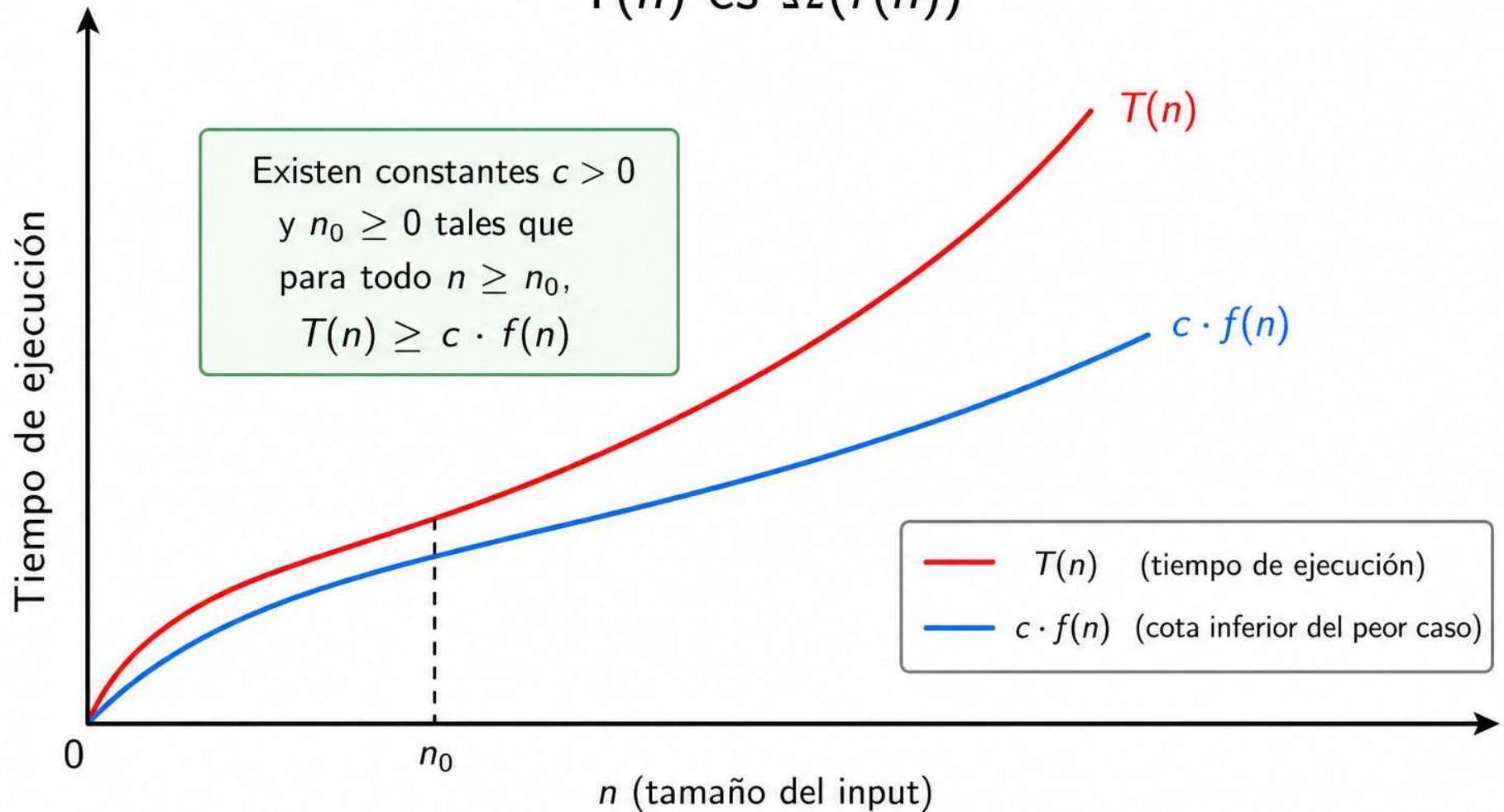
# Cota inferior del tiempo de un algoritmo A

Sea  $T_A(n)$  el tiempo de ejecución del peor caso de un algoritmo A definido sobre el tamaño  $n$  de la entrada de A.

$T_A(n) = \Omega(f(n))$  si y solo si existen  $c$  y  $n_0$  tales que  $T_A(n) \geq cf(n)$  para todo  $n \geq n_0$ .

Es decir  $f$  es una cota inferior del tiempo de A en el peor escenario.

$T(n)$  es  $\Omega(f(n))$



A partir de  $n_0$ , el tiempo de ejecución  $T(n)$  está siempre por encima de una constante multiplicada por  $f(n)$ . Por lo tanto,  $T(n)$  es  $\Omega(f(n))$ .

# Tiempo de un algoritmo

- El tiempo de ejecución de un algoritmo depende de la cantidad de operaciones primitivas realizadas.
- Las operaciones primitivas toman tiempo constante y son:
  - Asignar un valor a una variable
  - Invocar un método
  - Realizar una operación aritmética
  - Comparar dos números
  - Indexar un arreglo
  - Seguir una referencia de objeto
  - Retornar de un método
- Como estos tiempos dependen del compilador y del hardware subyacente, por ello los notaremos con constantes arbitrarias  $c_1, c_2, c_3, \dots$

# Reglas para calcular $T(n)$ a partir del código fuente de un algoritmo

Paso 1: Determinar el tamaño de la entrada  $n$

Paso 2: Aplicar las siguientes reglas en forma sistemática:

- $T_p(n) = \text{constante}$  si  $P$  es una acción primitiva
- $T_{S_1; \dots; S_k}(n) = T_{S_1}(n) + \dots + T_{S_k}(n)$
- $T_{\text{if } B \text{ then } S_1 \text{ else } S_2}(n) = T_B(n) + \max(T_{S_1}(n), T_{S_2}(n))$
- $T_{\text{for}(m; S)}(n) = m * T_S(n)$  donde  
 $m = \text{cant\_iteraciones}(\text{for}(m; S))$
- $T_{\text{while } B \text{ do } S}(n) = m * (T_B(n) + T_S(n)) + T_B(n)$  donde  
 $m = \text{cant\_iteraciones}(\text{while } B \text{ do } S)$
- $T_{\text{call } P}(n) = T_S(n)$  donde `procedure P; begin S end`
- $T_{f(e)}(n) = T_{x:=e}(n) + T_S(n)$  donde `function f(x); begin S end`

Ejemplo: Calcular el orden del tiempo de ejecución de sumar la primera y la última componentes de un arreglo a de n componentes.

Procedimiento:

```
public int sumar_pri_ult(int [] a, int n) {  
    int primera = a[0];  
    int ultima = a[n-1];  
    return primera + ultima;  
}
```

## Análisis de la complejidad algorítmica:

Entrada: a

Tamaño de la entrada: n

```
public int sumar_pri_ult(int [] a, int n) {  
    int primera = a[0];  c1 porque [] y = son primitivas  
    int ultima = a[n-1]; c2 idem  
    return primera + ultima; c3 idem  
}
```

Como es una secuencia, sumo los tiempos de las sentencias (o me quedo con el máximo de ellos).

$$T(n) = c_1 + c_2 + c_3 = c_4 = c_4 * 1 \text{ para } n_0=0$$

Entonces,  $T(n) = O(1)$

Nota: Quiere decir que el tiempo que toma es independiente del tamaño del arreglo.

Ejemplo: Calcular el orden del tiempo de ejecución de hallar el máximo entre la primera y la última componentes de un arreglo a de n componentes.

Procedimiento:

```
public int maximo_pri_ult(int [] a, int n) {  
    int primera = a[0];  
    int ultima = a[n-1];  
    int resultado;  
    if ( primera > ultima )  
        resultado = primera;  
    else  
        resultado = ultima;  
    return resultado;  
}
```

## Análisis de la complejidad algorítmica:

Entrada: a

Tamaño de la entrada: n

```
public int sumar_pri_ult(int [] a, int n) {  
    int primera = a[0];  c1 porque [] y = son primitivas  
    int ultima = a[n-1]; c2 idem  
    int resultado; en realidad esto no se ejecuta, es una definición de variable  
    if ( primera > ultima ) c3 porque > es primitiva  
        resultado = primera; c4 porque = es primitiva y acceder a variables también  
    else  
        resultado = ultima; c5 porque = es primitiva y acceder a variables también  
    return resultado; c6 porque return es primitiva y acceder a variables también  
}
```

Como es una secuencia, sumo los tiempos de las sentencias (o me quedo con el máximo de ellos). Para el if-then-else, el tiempo es el tiempo de la condición más el máximo entre el tiempo del bloque then y del bloque else.

$T(n) = c_1 + c_2 + c_3 + \max(c_4, c_5) + c_6 = c_7$  (que sería la suma de las constantes)

Entonces,  $T(n) = O(1)$

Nota: Quiere decir que el tiempo que toma es independiente del tamaño del arreglo.



Ejercicio: Mostrar que la búsqueda lineal en un arreglo de n enteros tiene orden n.

```
public static int lsearch( int [] a, int n, int x )
{
    int i = 0;
    boolean encuentre = false;
    while( i < n && !encuentre )
        if( a[i] == x )
            encuentre = true;
        else i++;
    if( encuentre ) return i;
    else return -1;
}
```

Tamaño de la entrada:  $n$  = cantidad de componentes de  $a$

Peor caso:  $x$  no está en  $a$

```
public static int lsearch( int [] a, int n, int x )
```

```
{
```

```
    int i = 0;
```

$c_1$

```
    boolean encuentre = false;
```

$c_2$

```
    while( i < n && !encontre )
```

Peor caso:  $n$  iteraciones

```
        if( a[i] == x )
```

Tiempo de condición:  $c_3$

```
            encuentre = true;
```

Tiempo del cuerpo:  $c_4$

```
            else i++;
```

```
    if( encuentre ) return i;
```

Tiempo de este if:  $c_5$

```
    else return -1;
```

```
}
```

$$T(n) = c_1 + c_2 + (n(c_3 + c_4) + c_3) + c_5 = O(n)$$

El tiempo es *lineal* en el tamaño de la entrada.

## Ejercicio: Versión de lsearch no estructurada

```
public static int lsearch( int [] a, int n, int x ) {  
    for ( int i = 0; i<n; i++ )           n iteraciones  
        if ( a[i] == x )                 c1  
            return i;                     c2  
    return -1;                             c3  
}
```

Por sintaxis:  $T(n) = n(c_1 + c_2) + c_3 = O(n)$

Este código también tiene  $O(n)$  sin embargo es más eficiente porque realiza menos comparaciones que el anterior.

Un compilador optimizante puede decidir almacenar  $i$  en un registro de la CPU y no en una variable haciendo el código más rápido todavía (disminuyendo la constante de proporcionalidad pero no el orden).

Ejercicio: Mostrar que la búsqueda binaria (dicotómica) en un arreglo de n componentes ordenadas en forma ascendente tiene orden logarítmico de base 2 en la cantidad de elementos del arreglo.

```
public static int bsearch( int [] a, int n, int x ) {  
    int ini = 0, fin = n-1;  
    while( ini <= fin ) {  
        int medio = (ini + fin) / 2;  
        if( a[medio] == x ) return medio;  
        else if( a[medio] > x ) fin = medio-1;  
        else ini = medio + 1;  
    }  
    return -1;  
}
```

Tamaño de la entrada:  $n$  = cantidad de componentes de  $a$   
Peor caso:  $x$  no está en  $a$

```
public static int bsearch( int [] a, int n, int x ) {  
    int ini = 0, fin = n-1;            $c_1$   
    while( ini <= fin ) {             Tiempo de la condición:  $c_2$   
        int medio = (ini + fin) / 2;   Tiempo del cuerpo:  $c_3$   
        if( a[medio] == x ) return medio;  
        else if( a[medio] > x ) fin = medio-1;  
        else ini = medio + 1;  
    }  
    return -1;                          $c_4$   
}
```

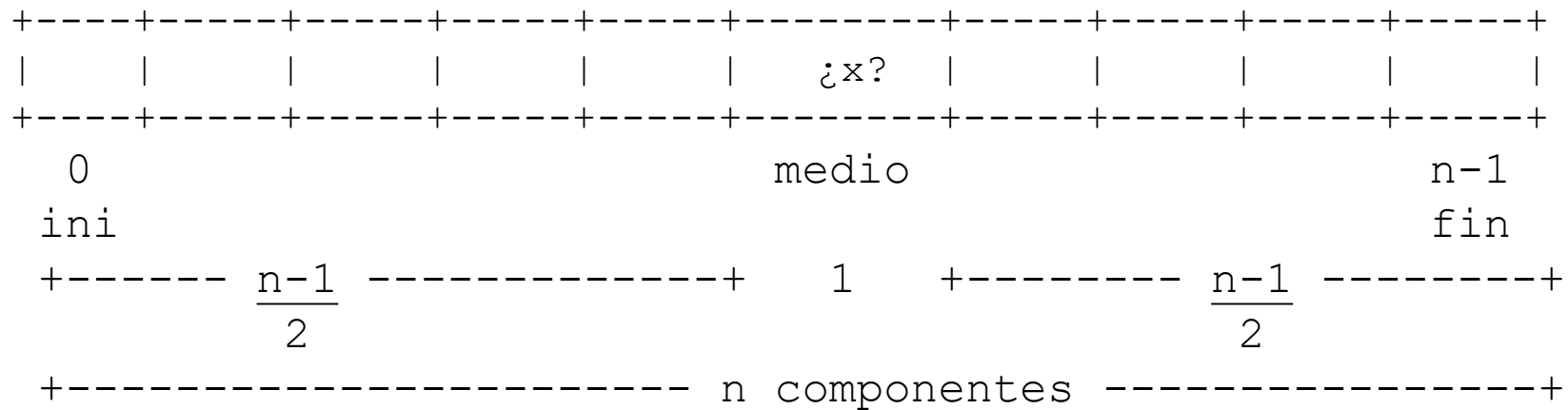
Sea  $k$  = cantidad de iteraciones del while, entonces

$$T(n) = c_1 + k(c_2 + c_3) + c_2 + c_4.$$

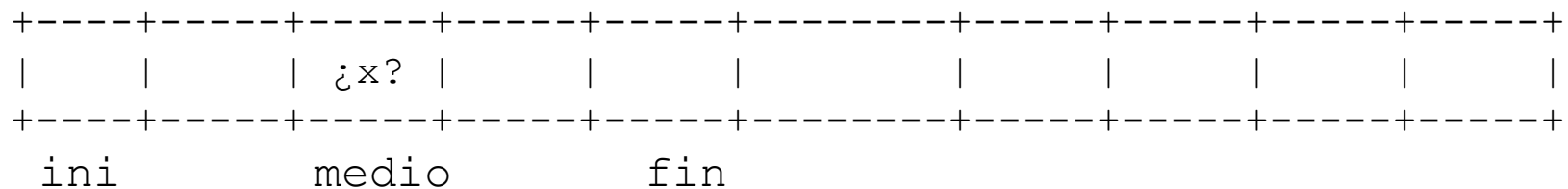
La pregunta es cómo definir  $k$  en función de  $n$ .

## Estimación de k:

Al partir el arreglo por la mitad ocurre esto:



Al partir de nuevo en la segunda iteración:



+-----+ ¿Cuánto mide la mitad de la mitad menos 1?

$$\frac{\frac{(n-1)}{2} - 1}{2} = \frac{\frac{(n-1)}{2} - 2}{2} = \frac{n - 1 - 2}{4} = \frac{n-3}{4}$$

## ¿Cómo estimar k en función de n?

El peor caso es cuando “x” no está en “a”.

Veamos cómo vamos descartando componentes del arreglo en función del número de iteración del while: Si tenemos n componentes, en cada iteración se descarta la componente del medio del arreglo, entonces de las n-1 componentes que falta revisar sólo se va considerar la mitad, entonces quedan  $(n-1)/2$  componentes para la siguiente iteración, y así sucesivamente.

Calculemos cuál es caso para la iteración genérica k.

| Número de iteración | Componentes por revisar             |
|---------------------|-------------------------------------|
| 1                   | n                                   |
| 2                   | $(n-1)/2$                           |
| 3                   | $(n-3)/4$                           |
| 4                   | $(n-7)/8$                           |
| 5                   | $(n-15)/16$                         |
| ...                 | ...                                 |
| k                   | $\frac{n - (2^{k-1} - 1)}{2^{k-1}}$ |

Entonces vimos que en la iteración  $k$ , la cantidad de componentes que quedan por revisar es:

$$\frac{n - (2^{k-1} - 1)}{2^{k-1}}$$

Como  $x$  no está en el arreglo  $a$ , en la última iteración completa que realiza el while queda una componente del arreglo por revisar.

Entonces:

$$\frac{n - (2^{k-1} - 1)}{2^{k-1}} = 1;$$

$$n - (2^{k-1} - 1) = 2^{k-1};$$

$$n - 2^{k-1} + 1 = 2^{k-1};$$

$$n + 1 = 2^{k-1} + 2^{k-1};$$

$$n + 1 = 2 \times 2^{k-1};$$

$$n + 1 = 2^k;$$

$$\log_2(n + 1) = k.$$

Con lo que  $T(n) = c_1 + \log_2(n+1)(c_2+c_3) + c_2 + c_4$ .

Luego, por regla de la suma,  $T(n) = O(\log_2(n))$ .



Búsqueda binaria/dicotómica con código estructurado (para puristas):

```
public static int bsearch( int [] a, int n, int x ) {  
    int ini = 0, fin = n-1;  
    int p = -1;  
    // p posición del elemento buscado, -1 = no encuentre, asumo que no está.  
  
    while( ini <= fin && p == -1 ) {  
        int medio = (ini + fin) / 2;  
        if( a[medio] == x ) p = medio;  
        else if( a[medio] > x ) fin = medio-1;  
        else ini = medio + 1;  
    }  
  
    return p;  
}
```

El análisis del tiempo es el mismo que para el código mostrado previamente.

Aunque la constante del testeo de la condición sería (muy levemente) mayor a la de la versión no estructurada.

Recuerden que un int en Java mide 4 bytes, entonces un arreglo de 1 terabyte tendría  $1024 \times 1024 \times 1024 \times 1024$  bytes / 4 bytes = 274.877.906.944 componentes y el logaritmo base 2 de eso es 38. O sea, el while hace 38 iteraciones en el peor caso.

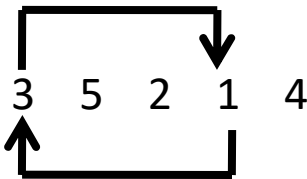
La búsqueda lineal haría 274.877.906.944 iteraciones en el peor caso.

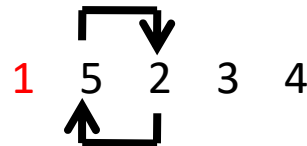
# Ordenamiento por selección

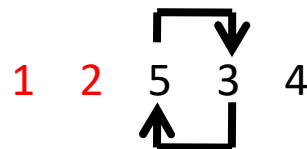
- Algoritmo:

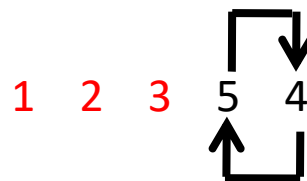
Haremos  $n-1$  pasadas sobre el arreglo  $a$

En la pasada  $i$ -ésima, encontramos el  $i$ -ésimo menor elemento del arreglo y lo intercambiamos con  $a[i]$

Pasada 1:  intercambiar  $a[0]$  con  $a[3]$

Pasada 2:  intercambiar  $a[1]$  con  $a[2]$

Pasada 3:  intercambiar  $a[2]$  con  $a[3]$

Pasada 4:  intercambiar  $a[3]$  con  $a[4]$

1 2 3 4 5

Ejercicio: Mostrar que el método de ordenamiento de arreglos por *selección* tiene orden cuadrático en la cantidad de elementos del arreglo.

```
public static void selection_sort( int []a, int n ) {  
    for( int i=0; i<n-1; i++ ) { // Repetir n-1 veces  
        int k = i; // Hallar k tq  $a_k$  es el mínimo de  
        for( int j=i+1; j<n; j++ ) //  $a_i, a_{i+1}, \dots, a_{n-1}$   
            if( a[j] < a[k] )  
                k = j;  
        swap( a, i, k ); // Intercambiar  $a_i$  con  $a_k$   
    }  
}
```

```

public static void selection_sort( int []a, int n ) {
    for( int i=0; i<n-1; i++ ) { n-1 iteraciones
        int k = i; c1
        for( int j=i+1; j<n; j++ ) n-1 iteraciones a lo sumo
            if( a[j] < a[k] ) k = j; c2
        swap( a, i, k ); c3
    }
}

```

Sea n = cantidad de elementos del arreglo

$$\begin{aligned}
 T(n) &= (n-1)T_{\text{cuerpo}}(n) \\
 &= (n-1) (c_1 + (n-1)T_{\text{cuerpo2}}(n) + c_3) \\
 &= (n-1) (c_1 + (n-1)c_2 + c_3) \\
 &= (n-1)c_1 + (n-1)^2c_2 + (n-1)c_3 \\
 &= (c_1+c_3)(n-1) + (n-1)^2c_2 \\
 &\leq c_5(n-1)^2 \\
 &\leq c_6n^2
 \end{aligned}$$

Po lo tanto,  $T(n)$  es  $O(n^2)$

# Análisis de tiempo alternativo

- Qué ocurre cuando noto que las iteraciones del bucle interior dependen del exterior:

```
public static void selection_sort( int []a, int n ) {  
    for( int i=0; i<n-1; i++ ) {  
        int k = i;     $c_1$   
        for( int j=i+1; j<n; j++ )  $j$  comienza en  $i+1$   
            if( a[j] < a[k] ) k = j;  
        swap( a, i, k );  
    }  
}
```

# Propiedades útiles

Para probar por inducción:

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

De las series:

$$\sum_{i=1}^n (a_i + b_i) = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$$

Informalmente:  $\Sigma$  distribuye en +

$$\sum_{i=1}^n c a_i = c \sum_{i=1}^n a_i$$

Informalmente: Las constantes se pueden sacar afuera de las  $\Sigma$ .

$$\sum_{i=1}^n (a_i + b_i) = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$$

$$\sum_{i=1}^n (a_i + b_i) = (a_1 + b_1) + (a_2 + b_2) + \dots + (a_n + b_n)$$

$$= a_1 + b_1 + a_2 + b_2 + \dots + a_n + b_n =$$

$$= a_1 + a_2 + \dots + a_n + b_1 + b_2 + \dots + b_n$$

$$= (a_1 + a_2 + \dots + a_n) + (b_1 + b_2 + \dots + b_n)$$

$$= \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$$

Tamaño de la entrada:  $n$  = cantidad de componentes de  $a$ .

```
public static void selection_sort( int []a, int n ) {  
    for( int i=0; i<n-1; i++ ) { // n-1 iteraciones (desiguales)  
        int k = i; //  $c_1$   
        for( int j=i+1; j<n; j++ ) //  $(n-1)-(i+1)+1$  iteraciones  
            if( a[j] < a[k] ) // Tiempo del if:  $c_2$   
                k = j;  
        swap( a, i, k ); // Tiempo de swap:  $c_3$   
    }  
}
```

$$T(n) = \sum_{i=0}^{n-2} (c_1 + ((n-1) - (i+1) + 1)c_2 + c_3)$$



$$T(n) = \sum_{i=0}^{n-2} (c_1 + ((n-1) - (i+1) + 1)c_2 + c_3) =$$

$$= \sum_{i=0}^{n-2} c_1 + c_2 \sum_{i=0}^{n-2} (n - i - 1) + \sum_{i=0}^{n-2} c_3$$

$$= c_1(n - 2 - 0 + 1) + c_2 \sum_{i=0}^{n-2} (n - i - 1) + c_3(n - 2 - 0 + 1)$$

$$= c_1(n - 1) + c_2 \sum_{i=0}^{n-2} (n - i - 1) + c_3(n - 1)$$

Falta ver cuánto vale el término del medio.

$$\begin{aligned}
& \sum_{i=0}^{n-2} (n - i - 1) = \\
& = (n - 1) + (n - 2) + (n - 3) + \cdots + (n - (n - 2) - 1) = \\
& = (n - 1) + (n - 2) + (n - 3) + \cdots + 1 = \\
& \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}
\end{aligned}$$

Entonces, sabíamos que:

$$\begin{aligned}
T(n) &= c_1(n - 1) + c_2 \sum_{i=0}^{n-2} (n - i - 1) + c_3(n - 1) = \\
&= c_1(n - 1) + c_2 \frac{n(n-1)}{2} + c_3(n - 1) = O(n^2)
\end{aligned}$$

# Resultados

- Si tengo un arreglo de  $n$  componentes:
- La búsqueda lineal tiene  $O(n)$
- La búsqueda binaria tiene  $O(\log(n))$
- Los métodos de ordenamiento multipasada (selección, burbuja, inserción) tienen  $O(n^2)$ .
- Ejemplo: Si el arreglo mide  $n=1000$  componentes:
  - Buscar linealmente toma  $\cong 1000$  pasos.
  - Buscar binariamente toma  $\log_2(1000) \cong 10$  pasos.
  - Ordenar el arreglo toma  $2(n-1) + n(n-1)/2 \cong 500000$  pasos.
- Conclusión: Si el arreglo está ordenado, conviene búsqueda binaria. Ordenar el arreglo es caro.

Problema: Determinar el tiempo de hallar el fondo de una pila.

Procedimiento:

```
public <E> E hallarFondo(Stack<E> pila) throws Exception {  
    if (pila.isEmpty())  
        throw new Exception("Pila Vacía");  
    else {  
        E resultado = pila.pop();  
        while ( pila.isEmpty() == false ) {  
            resultado = pila.pop();  
        }  
        return resultado;  
    }  
}
```

Entrada: pila

Tamaño del problema:  $n$  = cantidad de elementos de la pila

Antes de empezar tenemos que ver que determinar el tiempo de isEmpty y de pop.

Viendo el código de la implementación (de cualquiera de las dos pilas que hicimos), vemos que tienen  $O(1)$ .

```
public <E> E hallarFondo(Stack<E> pila) throws Exception {  
    if (pila.isEmpty())    c1  
        throw new Exception("Pila Vacía"); c2  
    else {  
        E resultado = pila.pop(); c3  
        while ( pila.isEmpty() == false ) { cond con tiempo c4, n-1 iteraciones  
            resultado = pila.pop(); c5  
        }  
        return resultado; c6  
    }  
}
```

$$T(n) = c1 + \max(c2, c3 + (c4+c5)(n-1) + c6) = c1 + \max(c2, k1n + k2) = O(n)$$

Ejemplo: Hallar el máximo entre el primer elemento y el último elemento de una lista simplemente enlazada de números:

```
public float maximo_pri_ult(  
    ListaSimplementeEnlazada<Float> lista ) {  
    float resultado;  
    float primero = lista.first().element();  
    float ultimo = lista.last().element();  
    if ( primero < ultimo ) resultado = primero;  
    else resultado = ultimo;  
    return resultado;  
}
```

Entrada: lista

Tamaño de la entrada:  $n$  = cantidad de elementos de lista

```
public float maximo_pri_ult( ListaSimplementeEnlazada<Float> lista ) {
```

```
    float resultado;  $c_1$ 
```

```
    float primero = lista.first().element();  $c_2$  porque first tiene  $O(1)$  y  
    element tiene  $O(1)$ 
```

```
    float ultimo = lista.last().element();  $c_3n$  porque last tiene  $O(n)$  porque  
    requiere recorrer la lista y element tiene  $O(1)$ 
```

```
    if ( primero < ultimo ) la condición tiene tiempo  $c_4$  porque es  
    primitiva
```

```
        resultado = primero; el then tiene tiempo  $c_5$  porque es primitiva  
    else
```

```
        resultado = ultimo; el else tiene tiempo  $c_6$  porque es primitiva  
    return resultado;  $c_7$  porque es primitiva
```

```
}
```

$$T(n) = c_1 + c_2 + c_3n + c_4 + \max(c_5, c_6) + c_7 = c_1 + c_2 + c_3n + c_4 + c_8 + c_7 = \max(c_1, c_2, c_3n, c_4, c_8, c_7) = O(n)$$

¿Y si la lista es DoblementeEnlazada con centinelas inicial y final?

Entrada: lista

Tamaño de la entrada:  $n$  = cantidad de elementos de lista

```
public float maximo_pri_ult( ListaDE<Float> lista ) {
```

```
    float resultado;  $c_1$ 
```

```
    float primero = lista.first().element();  $c_2$  porque first tiene  $O(1)$  y  
    element tiene  $O(1)$ 
```

```
    float ultimo = lista.last().element();  $c_3$  porque last tiene  $O(1)$  porque y  
    element tiene  $O(1)$ 
```

```
    if ( primero < ultimo ) la condición tiene tiempo  $c_4$  porque es  
    primitiva
```

```
        resultado = primero; el then tiene tiempo  $c_5$  porque es primitiva  
    else
```

```
        resultado = ultimo; el else tiene tiempo  $c_6$  porque es primitiva
```

```
    return resultado;  $c_7$  porque es primitiva
```

```
}
```

$T(n) = c_1 + c_2 + c_3 + c_4 + \max(c_5, c_6) + c_7 = c_1 + c_2 + c_3 + c_4 + c_8 + c_7 =$   
 $\max(c_1, c_2, c_3n, c_4, c_8, c_7) = O(1)$



# Qué ocurre con la tabla de hash?

# Tiempo de ejecución de hash abierto

- Entrada: Tabla de hash de N buckets y n entradas.
- Tamaño de la entrada: n = cantidad de entradas del hash
- Suponemos que no hay distribución uniforme de claves
- Peor caso:
  - Ocurre el peor escenario: Todas las claves terminan en un único bucket
  - En este caso el bucket es una lista de tamaño n
  - $T_{\text{get}}(n) = O(1+n)$ ,
  - $T_{\text{put}}(n) = O(1+n)$
  - $T_{\text{remove}}(n) = O(1+n)$
  - Nota: El “1” en “1+n” corresponde al tiempo de calcular el hash de la clave, el cual es independiente de n.

# Tiempo de ejecución de hash abierto

- Entrada: Tabla de hash de N buckets y n entradas.
- Tamaño de la entrada: n = cantidad de entradas del hash
- Suponemos que sí hay distribución uniforme de claves
- Peor caso:
  - Cada bucket tiene  $n/N$  entradas
  - En este caso el bucket es una lista de tamaño  $n/N$
  - $T_{\text{get}}(n) = O(1+n/N)$
  - $n/N = \lambda = \text{factor de carga} < 0,9$
  - Luego,  $T_{\text{get}}(n) = O(1+n/N) = O(1+\lambda) = O(1,9) = O(1)$
- Conclusión: Bajo distribución uniforme de claves y factor de carga mantenido, las operaciones de inserción, búsqueda y eliminación del hash tienen orden 1.

# Complejidad combinada

- Ocurre cuando tengo más de una entrada y el tiempo queda en función de más de un tamaño de entrada.
- Ejemplo: Dada una pila  $p$  y una cola  $q$  de números, determinar cuantos elementos de  $p$  son mayores al máximo de  $q$ .
- Algoritmo de solución:
  - 1) Hallar el máximo de  $q$  y llamarlo  $\max$
  - 2) Para cada elemento de  $p$ , contarlo si supera a  $\max$
  - 3) Retornar el contador
- Sea  $n$ =tamaño de  $p$  y  $m$ =tamaño de  $q$
- $T(n,m) = O(m) + O(n) + O(1) = O(n+m)$

# Bibliografía

- Goodrich & Tamassia. Data Structures and Algorithms. Fourth Edition. Capítulo 4.
- Goodrich, Tamassia & Goldwasser. Data Structures and Algorithms. Fourth Edition. Capítulo 4, pags 167-177.